

```

"""
Automated Web Scraping & Data Extraction Pipeline
-----
Target: https://books.toscrape.com (a public sandbox site built for scraping pra

Features:
    - Custom headers (polite User-Agent)
    - Rate limiting between requests
    - Retry logic with exponential backoff
    - HTML -> structured data (title, price, rating, availability)
    - Exports to CSV and JSON
    - Prints/saves a summary statistics report after each run

Usage:
    python scraper.py --pages 5 --delay 1.5
"""

import argparse
import csv
import json
import logging
import time
import statistics
from dataclasses import dataclass, asdict
from typing import List, Optional

import requests
from requests.adapters import HTTPAdapter, Retry
from bs4 import BeautifulSoup

BASE_URL = "https://books.toscrape.com/catalogue/page-{}.html"

HEADERS = {
    "User-Agent": (
        "Mozilla/5.0 (compatible; RabTechScraperBot/1.0; "
        "+https://example.com/bot-info) "
        "Python-requests educational-scraper"
    ),
    "Accept-Language": "en-US,en;q=0.9",
}

RATING_MAP = {"One": 1, "Two": 2, "Three": 3, "Four": 4, "Five": 5}

logging.basicConfig(

```

```

        level=logging.INFO,
        format="%(asctime)s [%(levelname)s] %(message)s",
    )
log = logging.getLogger("scraper")

```

```

@dataclass
class Book:
    title: str
    price_gbp: float
    rating: int
    availability: str
    page: int

```

```

def build_session(retries: int = 4, backoff: float = 0.6) -> requests.Session:
    """Create a requests Session with retry + backoff baked in."""
    session = requests.Session()
    session.headers.update(HEADERS)

    retry_strategy = Retry(
        total=retries,
        backoff_factor=backoff,
        status_forcelist=[429, 500, 502, 503, 504],
        allowed_methods=["GET"],
    )
    adapter = HTTPAdapter(max_retries=retry_strategy)
    session.mount("https://", adapter)
    session.mount("http://", adapter)
    return session

```

```

def fetch_page(session: requests.Session, url: str, timeout: int = 10) -> Optional[
    try:
        resp = session.get(url, timeout=timeout)
        resp.raise_for_status()
        return resp.text
    except requests.RequestException as e:
        log.warning(f"Failed to fetch {url}: {e}")
        return None

```

```

def parse_page(html: str, page_num: int) -> List[Book]:
    soup = BeautifulSoup(html, "html.parser")
    books = []
    for article in soup.select("article.product_pod"):
        title = article.h3.a["title"].strip()

```

```

price_text = article.select_one(".price_color").get_text(strip=True)
price = float(price_text.replace("£", "").replace("Â", ""))

rating_class = article.select_one("p.star-rating")["class"]
rating_word = [c for c in rating_class if c != "star-rating"][0]
rating = RATING_MAP.get(rating_word, 0)

availability = article.select_one(".availability").get_text(strip=True)

books.append(
    Book(
        title=title,
        price_gbp=price,
        rating=rating,
        availability=availability,
        page=page_num,
    )
)
return books

```

```

def scrape(pages: int, delay: float) -> List[Book]:
    session = build_session()
    all_books: List[Book] = []

    for page_num in range(1, pages + 1):
        url = BASE_URL.format(page_num)
        log.info(f"Fetching page {page_num}: {url}")
        html = fetch_page(session, url)
        if html is None:
            log.warning(f"Skipping page {page_num} after failed retries.")
            continue

        page_books = parse_page(html, page_num)
        log.info(f" -> extracted {len(page_books)} items")
        all_books.extend(page_books)

        if page_num < pages:
            time.sleep(delay) # rate limiting between requests

    return all_books

```

```

def export_csv(books: List[Book], path: str = "output.csv") -> None:
    if not books:
        log.warning("No data to write to CSV.")

```

```

        return
    with open(path, "w", newline="", encoding="utf-8") as f:
        writer = csv.DictWriter(f, fieldnames=list(asdict(books[0]).keys()))
        writer.writeheader()
        for b in books:
            writer.writerow(asdict(b))
    log.info(f"Wrote {len(books)} rows to {path}")

def export_json(books: List[Book], path: str = "output.json") -> None:
    with open(path, "w", encoding="utf-8") as f:
        json.dump([asdict(b) for b in books], f, indent=2)
    log.info(f"Wrote {len(books)} records to {path}")

def summary_report(books: List[Book], path: str = "summary_report.txt") -> None:
    if not books:
        log.warning("No data available for summary report.")
        return

    prices = [b.price_gbp for b in books]
    ratings = [b.rating for b in books]

    lines = [
        "===== SCRAPE SUMMARY REPORT =====",
        f"Total items scraped : {len(books)}",
        f"Pages covered      : {sorted(set(b.page for b in books))}",
        f"Average price (GBP) : {statistics.mean(prices):.2f}",
        f"Min / Max price     : {min(prices):.2f} / {max(prices):.2f}",
        f"Average rating      : {statistics.mean(ratings):.2f} / 5",
        f"Rating distribution : { {r: ratings.count(r) for r in sorted(set(ratin
        f"In-stock items   : {sum(1 for b in books if 'In stock' in b.availab
        "=====",
    ]
    report = "\n".join(lines)
    print(report)
    with open(path, "w", encoding="utf-8") as f:
        f.write(report + "\n")
    log.info(f"Saved summary report to {path}")

def main():
    parser = argparse.ArgumentParser(description="ETL scraper for books.toscrape.
    parser.add_argument("--pages", type=int, default=5, help="Number of catalogue
    parser.add_argument("--delay", type=float, default=1.5, help="Delay in second
    args = parser.parse_args()
```

```
log.info(f"Starting scrape: {args.pages} pages, {args.delay}s delay between r
books = scrape(args.pages, args.delay)

export_csv(books)
export_json(books)
summary_report(books)

if __name__ == "__main__":
    main()
```